เอกสารสรุปสถาปัตยกรรมและการสื่อสารข้อมูล (System Communication Blueprint)

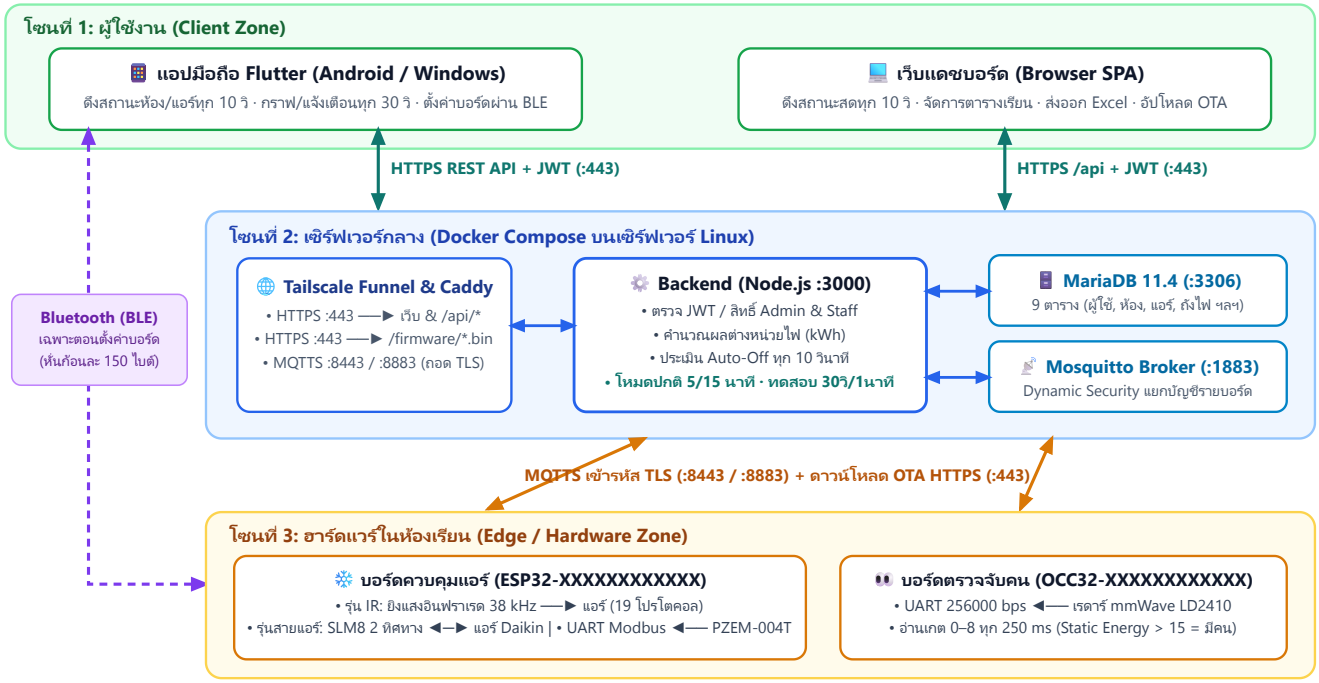
ภาพรวมการสื่อสารทั้งระบบ RMUTT Save Energy (อาคาร 16)

อธิบายละเอียดตั้งแต่ระดับเซนเซอร์และบอร์ด ESP32 หน้าห้องเรียน ไปจนถึง MQTT Broker, Backend API, ฐานข้อมูล MariaDB, แอปมือถือ Flutter และเว็บแดชบอร์ด — ส่งอะไร รับอะไร ใช้โปรโตคอลไหน ตั้งแต่ต้นจนจบ

แอป Flutter (Android / Windows) | เว็บแดชบอร์ด (SPA) | Backend Node.js + MariaDB | Mosquitto MQTTS + Dynamic Security | บอร์ดแอร์ IR & สายแอร์ Daikin SLM8 + PZEM-004T | บอร์ดตรวจจับคน LD2410

ส่วนที่ 1 แผนผังภาพใหญ่ทั้งระบบ (Master Architecture & Communication Map)

ระบบถูกออกแบบเป็น **3 โซนแยกจากกันอย่างชัดเจน** โดยยึดหลักความปลอดภัยสำคัญคือ **แอปมือถือและเว็บแดชบอร์ดจะไม่คุยกับบอร์ด ESP32 หรือฐานข้อมูล MariaDB โดยตรง** (ยกเว้นขั้นตอนตั้งค่าบอร์ดครั้งแรกผ่าน Bluetooth ระยะใกล้) ทุกคำสั่งและทุกข้อมูลต้องวิ่งผ่าน **Backend** เพื่อตรวจสอบสิทธิ์ (JWT) ตรวจสอบความถูกต้องของข้อมูล และบันทึกเป็นข้อมูลจริงชุดเดียวที่ศูนย์กลาง



ตารางสรุปทุกช่องทางการสื่อสารในระบบ (11 เส้นทางหลัก)

ต้นทาง	ปลายทาง	โปรโตคอล / พอร์ต	จังหวะการรับ–ส่ง	สิ่งที่ส่งและรับ	ไฟล์โค้ดหลัก
แอป Flutter	**Backend**	HTTPS REST API (`:443`) + JSON + JWT	เมื่อกดปุ่ม + ดึงซ้ำทุก 10 วิ / 30 วิ	ล็อกอิน, ดูสถานะห้อง/แอร์, สั่งรีโมตแอร์, ดูกราฟพลังงาน, ตั้งค่าระบบ/โหมดทดสอบ	`lib/services/*_service.dart`
เว็บแดชบอร์ด	**Backend**	HTTPS `/api/*` (`:443`) + JSON + JWT	เมื่อกดปุ่ม + ดึงซ้ำทุก 10 วินาที	เหมือนแอปมือถือ + อัปโหลดไฟล์เฟิร์มแวร์ `.bin` และสั่งอัปเดต OTA หลายเครื่อง	`webapp/assets/js/api.js`
แอป Flutter	**บอร์ด ESP32**	Bluetooth Low Energy (BLE GATT)	เฉพาะตอนติดตั้งหรือตั้งค่าบอร์ดใหม่	อ่านตัวตนบอร์ด, สั่งบอร์ดสแกน Wi-Fi, ส่งรหัส Wi-Fi + บัญชี MQTT (ห้นก้อนละ 150 ไบต์)	`ble_provisioning_service.dart`
บอร์ดแอร์ ESP32	**Mosquitto Broker**	MQTTS เข้ารหัส TLS (`:8443` หรือ `:8883`)	ส่ง `/status` ทุก 10 วิ + ทันทีเมื่อเปลี่ยน	ส่งค่าไฟสด (V, A, W, kWh), รับคำสั่งแอร์ (`/command`), ส่งข้อมูลไฟย้อนหลัง (`/history`)	`rmutt_esp32_air.ino`
บอร์ดตรวจจับคน	**Mosquitto Broker**	MQTTS เข้ารหัส TLS (`:8443` หรือ `:8883`)	ส่ง `/status` ทุก 10 วิ + ทันทีเมื่อคนเข้า/ออก	ส่งสถานะมีคน/ไม่มีคน (`occupied`) และค่าพลังงานเรดาร์ทั้ง 9 เกต (`Gate0` – `Gate8`)	`rmutt_esp32_occupancy.ino`
บอร์ด ESP32 ทั้งคู่	**Caddy Server**	HTTPS GET `/firmware/*.bin` (`:443`)	เมื่อได้รับคำสั่ง `action: "ota"`	ดาวน์โหลดไฟล์ไบนารีเฟิร์มแวร์ใหม่มาเขียนลงพาร์ติชันสำรอง (`app0` / `app1`) แล้วรีบูต	`ota_update.h` , `routes/firmware.js`
Backend	**Mosquitto Broker**	MQTT ภายใน Docker (`mqtt:1883`)	เชื่อมต่อค้างตลอดเวลา (Persistent)	Subscribe รับข้อมูลจากทุกบอร์ด, Publish คำสั่งแอร์, จัดการบัญชี MQTT รายบอร์ดผ่าน `$CONTROL`	`backend/src/services/mqtt.js`
Backend	**MariaDB**	MariaDB TCP Protocol (`db:3306`)	ทุกครั้งที่มี API หรือข้อความ MQTT	อ่าน/เขียนตารางทั้ง 9 ตาราง และใช้ Advisory Lock (`GET_LOCK`) กันประเมิน Auto-off ซ้ำ	`backend/src/db.js`
บอร์ดแอร์ (รุ่น IR)	**เครื่องแอร์**	แสงอินฟราเรด `38 kHz` (ขา GPIO 4)	เมื่อได้รับคำสั่งจาก MQTT หรือตารางรายวัน	ส่งพัลส์รหัส IR ตามยี่ห้อแอร์ (สื่อสารทางเดียว แอร์ไม่ส่งข้อมูลกลับ)	`IRremoteESP8266` , `SaijoDenkiAC.cpp`

ต้นทาง	ปลายทาง	โปรโตคอล / พอร์ต	จังหวะการรับ–ส่ง	สิ่งที่ส่งและรับ	ไฟล์โค้ดหลัก
บอร์ดแอร์ (รุ่นสาย)	**แอร์ Daikin**	โปรโตคอลสายรีโมต SLM8 (RX=18, TX=19)	รับ–ส่งเฟรมต่อเนื่อง 2 ทิศทาง	รับ–ส่งเฟรม 8 ไบต์ + Checksum สั่งงานแอร์และรู้ทันทีเมื่อมีคนกดปุ่มที่รีโมตติดผนัง	`rmutt_esp32_air_wired.ino`
บอร์ด ESP32	**PZEM / LD2410**	UART (Modbus `9600` / เรดาร์ `256000`)	PZEM ทุก 2 วิ · LD2410 ทุก 250 ms	อ่านค่าไฟฟ้าจากสายไฟแอร์ และอ่านค่าการหายใจ/ขยับตัวของคนในห้อง	เฟิร์มแวร์ทั้ง 3 รุ่น

ส่วนที่ 2 เจาะลึกระดับฮาร์ดแวร์หน้างาน (ESP32 คุยกับเซนเซอร์และเครื่องแอร์อย่างไร)

1. บอร์ดแอร์ ◄——► มิเตอร์ไฟ PZEM-004T `UART Modbus 9600 bps`

ต่อผ่านขา `GPIO 16 (RX2)` และ `GPIO 17 (TX2)` บอร์ดส่งเฟรม Modbus ไปถามทุก **2 วินาที**:

- **สิ่งที่ส่งไป:** คำสั่งอ่าน Input Registers ของ PZEM
- **สิ่งที่รับกลับมา (6 ค่า):**
  - แรงดันไฟฟ้า (`voltage` , โวลต์ เช่น `229.4 V`)
  - กระแสไฟฟ้า (`current` , แอมแปร์ เช่น `4.15 A`)
  - กำลังไฟฟ้าจริง (`power` , วัตต์ เช่น `912.0 W`)
  - พลังงานสะสม (`energy` , หน่วย `kWh` เช่น `145.280 kWh`)
  - ความถี่ (`frequency` , `50.0 Hz`) และตัวประกอบกำลัง (`pf`)
- **การตัดสินว่าแอร์ทำงานจริง (Hysteresis 3 ครั้งติดกัน):**
  - วัดได้ `≥ 30 W` ติดกัน 3 รอบ (6 วินาที) ——► `acRunning = true`
  - วัดได้ `≤ 10 W` ติดกัน 3 รอบ (6 วินาที) ——► `acRunning = false`

2. บอร์ดตรวจจับคน ◄—— เรดาร์ LD2410 `UART 256000 bps`

ต่อผ่านขา `GPIO 16 (RX2)` และ `GPIO 17 (TX2)` ในโหมด Engineering Mode อ่านข้อมูลทุก **250 มิลลิวินาที (4 ครั้ง/วินาที)**:

- **สิ่งที่รับมาจากเรดาร์:**
  - สถานะเป้าหมายและระยะทาง (เซนติเมตร)
  - ค่าพลังงานสะท้อนแยกตามระยะ **เกต 0 ถึง เกต 8** (เกตละ ~0.75 เมตร รวม ~6.75 เมตร) ทั้งค่าเคลื่อนไหวและค่าอยู่นิ่ง (`staticEnergy` ที่จับการขยับอกขณะหายใจได้)
- **เกณฑ์ตัดสินว่ามีคนในห้อง:**
  - ตรวจ `Gate 0` ถึง `Gate 8` ถ้ามีเกตใดเกตหนึ่งที่ `staticEnergy > 15` ——► ถือว่า `occupied = true` **(มีคนอยู่)**
  - ถ้าเปลี่ยนจากมีคนเป็นไม่มีคน (หรือกลับกัน) บอร์ดจะส่ง MQTT `/status` แจ้งเซิร์ฟเวอร์ทันทีโดยไม่รอรอบ 10 วินาที

3. บอร์ดแอร์รุ่น IR ——► เครื่องปรับอากาศ `Infrared 38 kHz`

ส่งสัญญาณออกทางขา `GPIO 4` ผ่านวงจรทรานซิสเตอร์ขับหลอด IR LED:

- **สิ่งที่ส่งออกไป:** แปลงสถานะทั้งก้อน (เปิด/ปิด, อุณหภูมิ 16–30°C, โหมด, ความเร็วพัดลม, บานสวิง) เป็นชุดพัลส์แสงอินฟราเรด 38 kHz ตามยี่ห้อแอร์ (รองรับ 19 โปรโตคอล เช่น `DAIKIN` , `MITSUBISHI_AC` , `GREE` และคลาส `SaijoDenkiAC` ที่ถอดรหัสเฟรมสร้างเอง)
- **ข้อจำกัดสำคัญ:** อินฟราเรดเป็นการส่ง **ทางเดียว (One-way)** แอร์ไม่ส่งสัญญาณตอบกลับ ระบบจึงต้องดูค่าวัตต์จาก PZEM-004T เพื่อพิสูจน์ว่าแอร์เปิดหรือดับจริง

4. บอร์ดสายแอร์ Daikin ◄——► เครื่องแอร์ `โปรโตคอลสาย SLM8`

ต่อขนานเข้ากับสายรีโมตติดผนังของแอร์ Daikin (โปรโตคอล SLM8):

- **สิ่งที่รับและส่ง:** เฟรมข้อมูลขนาด **8 ไบต์** พร้อมไบต์ Checksum ท้ายเฟรม สื่อสารแบบ **2 ทิศทาง (Two-way)**
- **ข้อดีเหนือกว่า IR:** สั่งเปิด/ปิด, อุณหภูมิ 16–30°C, โหมด Cool/Dry/Fan, พัดลม Low/Med/High/Auto, สวิง และโหมด Sleep ได้โดยไม่กลัวสิ่งกีดขวางบังหลอด IR และรู้สถานะจริงจากสายสัญญาณโดยตรง

ส่วนที่ 3 การตั้งค่าบอร์ดครั้งแรกผ่าน Bluetooth Low Energy (BLE GATT Provisioning)

ตอนนำบอร์ด ESP32 ตัวใหม่ไปติดที่ห้อง บอร์ดยังไม่รู้ชื่อ Wi-Fi รหัสผ่าน และที่อยู่เซิร์ฟเวอร์ แอดมินจึงใช้แอปมือถือ Flutter คุยกับบอร์ดโดยตรงผ่าน **Bluetooth Low Energy (BLE)** ภายใต้ Service UUID หลัก `7b570001-a243-4e5b-8f13-179af81b2457`

ชื่อช่อง (Characteristic)	UUID	ทิศทาง	ข้อมูล JSON ที่รับ–ส่ง และหน้าที่
1. Identity (ตัวตนบอร์ด)	`7b570004-a243-4e5b-...`	แอปอ่าน (Read)	บอร์ดส่งตัวตนกลับมา: `{"deviceId":"ESP32-A1B2C3D4E5F6","bleName":"RMUTT-AC-A1B2C3D4E5F6","deviceType":"air","firmwareVersion":"2.5.5","configured":false}` แอปตรวจว่าชื่อบลูทูธตรงกับเลข MAC จริงของชิปหรือไม่
2. Settings (ค่าที่จำไว้)	`7b570006-a243-4e5b-...`	แอปอ่าน (Read)	ถ้าบอร์ดเคยตั้งค่าแล้ว (`configured: true`) แอปจะอ่านค่าเดิม (เช่น ชื่อ Wi-Fi เดิม, ห้องเดิม, ยี่ห้อแอร์เดิม) มาเติมลงในฟอร์มให้อัตโนมัติ ไม่ต้องพิมพ์ใหม่ทั้งหมด
3. Wi-Fi Scan (สแกนไวไฟ)	`7b570005-a243-4e5b-...`	แอปอ่าน (Read)	เมื่อแอปอ่านช่องนี้ บอร์ดจะสแกนหาคลื่น Wi-Fi 2.4 GHz รอบตัวบอร์ดจริง แล้วส่งกลับ: `{"ok":true,"networks":[["RMUTT-IoT",-55,true],["CPE-Lab",-68,true]]}` (ชื่อ, ความแรงสัญญาณ dBm, มีรหัสผ่านไหม)
4. Config Write (เขียนค่าตั้ง)	`7b570002-a243-4e5b-...`	แอปเขียน (Write)	แอปแปลง JSON ตั้งค่าทั้งหมดเป็นไบต์ UTF-8 แล้ว **หั่นเป็นก้อนละไม่เกิน 150 ไบต์** โดยแต่ละก้อนมีหัวแพ็กเก็ต 6 ไบต์: `[1, 1, ลำดับก้อน(สูง), ลำดับก้อน(ต่ำ), จำนวนก้อนรวม(สูง), จำนวนก้อนรวม(ต่ำ)]` ตามด้วยเนื้อข้อมูล เพื่อไม่ให้เกินเพดาน MTU ของ Bluetooth
5. Status Notify (ใบตอบรับ)	`7b570003-a243-4e5b-...`	บอร์ดแจ้ง (Notify)	หลังบอร์ดรับครบทุกก้อน บอร์ดจะลองต่อ Wi-Fi หันที แล้วส่งผลกลับมาทาง Notify: `{"ok":true,"wifiConnected":true,"saved":true,"message":"Configuration saved"}`

```
// ตัวอย่าง JSON ที่แอป Flutter ส่งเข้าช่อง Config Write (...0002) ไปยังบอร์ดแอร์
{
  "deviceId": "ESP32-A1B2C3D4E5F6",
  "roomNumber": "16301",
  "airNumber": 1,
  "protocol": "DAIKIN",
  "wifiSsid": "RMUTT-IoT",
  "wifiPassword": "********",
  "mqttHost": "rmutt-energy.tailb292b4.ts.net",   // ได้มาจาก Backend ตอนจองช่อง (POST /reservations)
  "mqttPort": 8443,
  "mqttUsername": "ac_esp32_a1b2c3d4e5f6",        // บัญชีเฉพาะของบอร์ดตัวนี้ตัวเดียว
  "mqttPassword": "********************************", // คำนวณจาก HMAC-SHA256(MQTT_DEVICE_SECRET, deviceId)
  "provisionToken": "9f86d081884c7d659a2feaa0...", // โทเคนใช้ครั้งเดียว อายุ 15 นาที สำหรับยืนยันตัวตนผ่าน MQTTS
  "autoOn": "08:00",
  "autoOff": "20:00"
}
```

ส่วนที่ 4 การสื่อสารระหว่างบอร์ด ESP32 กับเซิร์ฟเวอร์ผ่าน MQTT (MQTTS / Broker Layer)

เมื่อบอร์ดต่อ Wi-Fi สำเร็จ จะเชื่อมต่อเข้าสู่ **Mosquitto Broker** ด้วยโปรโตคอล **MQTTS (MQTT over TLS)** โดย Mosquitto ใช้ปลั๊กอิน **Dynamic Security** กันสิทธิ์ (ACL) ให้บอร์ดแต่ละตัวอ่านและเขียนได้เฉพาะ Topic ที่มีรหัส `deviceId` ของตัวเองเท่านั้น และตั้งค่า **Last Will and Testament (LWT)** ไว้ที่ Broker คือถ้าบอร์ดไฟดับหรือเน็ตหลุดเกินเวลา Keep-Alive ตัว Broker จะประกาศข้อความ `{"online":false}` เข้า Topic `/status` แทนบอร์ดทันที

4.1 ตาราง MQTT Topics ทั้งหมดของบอร์ดแอร์ (`rmutt/ac/<deviceId>/...`)

MQTT Topic	ทิศทาง	QoS / Retain	ส่งอะไร / รับอะไร และทำงานอย่างไร
`rmutt/ac/<id>/provision`	บอร์ด ⟶ เซิร์ฟเวอร์	QoS 1	**บอร์ดส่ง:** `{"token":"...","firmwareVersion":"2.5.5","bleName":"RMUTT-AC-...","roomNumber":"16301","airNumber":1,"protocol` ส่งหัน ทีหลังตั้งค่า BLE เสร็จ Backend นำ `token` ไปเทียบกับค่า Hash ในตาราง `air_conditioners` ถ้าตรงและยังไม่หมดอายุ 15 นาที จะเปลี่ยนสถานะเป็น `activ`
`rmutt/ac/<id>/provision-ack`	เซิร์ฟเวอร์ ⟶ บอร์ด	QoS 1	**เซิร์ฟเวอร์ตอบ:** `{"ok":true,"deviceId":"ESP32-...","roomNumber":"16301","airNumber":1}` เมื่อบอร์ดได้รับข้อความนี้ จะลบ `provisionToken` ออกจากหน่วยความจำ NVS ทันที และเริ่มทำงานปกติ
`rmutt/ac/<id>/status`	บอร์ด ⟶ เซิร์ฟเวอร์	Retained	**บอร์ดส่งทุก 10 วินาที (และทันทีเมื่อสถานะเปลี่ยน):** `{"online":true,"pzemAvailable":true,"acRunning":true,"voltage":229.4,"current":4.12,"power":910.5,"energy":145.28,"fr` Backend นำไปคำนวณผลต่างหน่วยไฟลงตาราง `energy_hourly` อัปเดตตาราง `air_conditioners` และกระตุ้น `evaluateAutoOff()`
`rmutt/ac/<id>/state`	บอร์ด ⟶ เซิร์ฟเวอร์	Retained	**บอร์ดส่งหลังยิงคำสั่งแอร์เสร็จ:** `{"power":true,"temperature":25,"mode":"cool","fanSpeed":"auto","swing":"off","horizontalSwing":false,"eco":false,"tur 1234"}` ทำให้แอปและเว็บรู้ว่าหน้าปัดรีโมตล่าสุดอยู่ที่ค่าอะไร
`rmutt/ac/<id>/command`	เซิร์ฟเวอร์ ⟶ บอร์ด	QoS 1	**เซิร์ฟเวอร์ส่งคำสั่งได้ 4 รูปแบบ:** 1) **สั่งรีโมตแอร์:** `{"commandId":"...","power":false,"temperature":25,"mode":"cool",...}` (จากผู้ใช้กด หรือจากระบบ Auto-Off) 2) **สั่งอัปเดต OTA:** `{"action":"ota","url":"https://.../firmware/rmutt_esp32_air-2.5.5.bin"}` 3) **สั่งเปิดบลูทูธซ้ำ:** `{"action":"enter_setup"}` 4) **สั่งถอดบอร์ดออกจากห้อง:** `{"action":"unprovision"}` (บอร์ดจะล้าง NVS แล้วกลับเข้าโหมด BLE)
`rmutt/ac/<id>/schedule`	เซิร์ฟเวอร์ ⟶ บอร์ด	Retained / QoS 1	**เซิร์ฟเวอร์ส่ง:** `{"configured":true,"withinSchedule":true,"termStatus":"active","updatedAt":"..."}` บอกบอร์ดว่าห้องนี้ตั้งตารางเรียนไว้ในระบบกลางแล้วหรือยัง และตอนนี้มีคาบเรียนหรือไม่
`rmutt/ac/<id>/history`	บอร์ด ⟶ เซิร์ฟเวอร์	QoS 1	**บอร์ดส่งข้อมูลย้อนหลังหลังเน็ตกลับมา:** `{"records":[{"ts":1729200000,"kwh":144.80,"running":1,"samples":150}, ...]}` ส่งทีละชุด ชุดละไม่เกิน 10 รายการ (แต่ละรายการคือสรุปทุก 5 นาทีที่บอร์ดเก็บลงพาร์ติชัน `rmuttlog` ตอนเน็ตหลุด)
`rmutt/ac/<id>/history-ack`	เซิร์ฟเวอร์ ⟶ บอร์ด	QoS 1	**เซิร์ฟเวอร์ตอบรับ:** `{"upTo":1729202700,"stored":10,"skipped":0}` ยืนยันว่าเซิร์ฟเวอร์บันทึกข้อมูลถึงเวลา `upTo` ลงฐานข้อมูลเรียบร้อยแล้ว บอร์ดจึงเลื่อนตัวชี้ Ring Buffer ลบชุดนั้นทิ้งแล้วส่งชุดถัดไป

4.2 ตาราง MQTT Topics ทั้งหมดของบอร์ดตรวจจับคน (`rmutt/occupancy/<deviceId>/...`)

MQTT Topic	ทิศทาง	QoS / Retain	ส่งอะไร / รับอะไร และทำงานอย่างไร
`rmutt/occupancy/<id>/provision` `.../provision-ack`	สองทิศทาง	QoS 1	ยืนยันตัวตนหลังตั้งค่า BLE เหมือนบอร์ดแอร์ (ผูก 1 ห้อง ต่อ 1 เซนเซอร์)
`rmutt/occupancy/<id>/status`	บอร์ด ⟶ เซิร์ฟเวอร์	Retained	**บอร์ดส่งทุก 10 วินาที และทันทีที่สถานะมีคนเปลี่ยน:** `{"online":true,"radarAvailable":true,"occupied":false,"Gate0":3,"Gate1":8,"Gate2":4,"Gate3":0,"Gate4":0,"Gate5` Backend อัปเดตตาราง `occupancy_sensors` และเรียก `evaluateAutoOff({roomNumber})` ทันที
`rmutt/occupancy/<id>/command`	เซิร์ฟเวอร์ ⟶ บอร์ด	QoS 1	รองรับคำสั่ง `{"action":"ota","url":"..."}` , `{"action":"enter_setup"}` และ `{"action":"unprovision"}`

> 💡 **ทำไมต้องแยกฟิลด์ radarAvailable ออกจาก occupied? (หลักการ Fail-Safe)**
> หากสายสัญญาณ UART ระหว่าง ESP32 กับเรดาร์ LD2410 หลุดหรือเรดาร์เสีย ค่าพลังงานทุกเกตจะเป็น `0` ถ้าบอร์ดส่งแค่ `occupied: false` ระบบจะเข้าใจผิดว่าห้องไม่มีคนแล้วสั่งปิดแอร์ใส่คนที่กำลังนั่งเรียนอยู่ ดังนั้นบอร์ดจึงส่ง `radarAvailable: false` มาด้วย และ Backend จะบังคับว่า **ถ้าเรดาร์ไม่ออนไลน์หรืออ่านค่าไม่ได้ ห้ามปิดแอร์เด็ดขาด**

ส่วนที่ 5 การสื่อสารระหว่างแอปมือถือ / เว็บแดชบอร์ด กับ Backend (HTTPS REST API + JWT)

ทั้งแอป Flutter (`lib/services/*_service.dart`) และเว็บแดชบอร์ด (`webapp/assets/js/api.js`) สื่อสารกับ Backend ผ่าน **HTTPS REST API** โดยแนบ `Authorization: Bearer <JWT>` ไปกับทุกคำขอ และมีการดึงข้อมูลซ้ำอัตโนมัติ (Polling) เป็นจังหวะเพื่อให้หน้าจอแสดงข้อมูลล่าสุดเสมอ:

จังหวะดึงข้อมูลอัตโนมัติทุก 10 วินาที Real-time Control

- `GET /api/rooms` — ดึงสถานะห้อง, มีคน/ไม่มีคน, และนาฬิกานับถอยหลังปิดแอร์อัตโนมัติ
- `GET /api/air-conditioners` — ดึงสถานะแอร์ทุกเครื่อง, กำลังไฟ (W), พลังงานสะสม (kWh), และสถานะออนไลน์

จังหวะดึงข้อมูลอัตโนมัติทุก 30 วินาที Dashboard & Alerts

- `GET /api/energy/summary` — ดึงยอดพลังงานรวมและจุดกราฟแท่งตามช่วงเวลาที่เลือก (วันนี้ / 7 วัน / 30 วัน / กำหนดเอง)
- `GET /api/alerts` — ดึงรายการแจ้งเตือนระบบ และเด้งแจ้งเตือนบนจอมือถือทันทีเมื่อพบใบแจ้งเตือนใหม่

ตารางสรุป REST API ทั้งหมด (ส่งอะไร ⟶ รับอะไร)

Method & Endpoint	สิทธิ์	ข้อมูลที่ส่งไป (Request Body / Query)	ข้อมูลที่รับกลับมา (Response JSON)
`POST /api/auth/login`	ทุกคน	`{"username":"admin","password":"..."}`	`{"token":"eyJ...","user":{"username","firstName","lastName","ema`

`POST /api/auth/forgot-password`
`POST /api/auth/verify-otp`
`POST /api/auth/reset-password`
	ทุกคน	1) `{"identifier":"ชื่อหรืออีเมล"}`
		2) `{"requestId":"...","otp":"123456"}`
		3) `{"resetToken":"...","newPassword":"..."}`
			ส่งรหัส OTP 6 หลักเข้าอีเมลผ่าน SMTP ⟶ ตรวจ OT
			เตะ Token เก่าทั้งหมดออกจากระบบ

`GET /api/rooms` `POST / PUT / DELETE /api/rooms`	อ่าน: ทุกคน แก้: Admin	ตอนสร้าง/แก้: `{"number":"16301","floor":3,"roomIndex":1,"type":"Lecture Room"}`	คืนรายการห้องพร้อม `isOccupied` , `occupancyS` (`idle` / `counting` / `commanding`), `autoOf`
`GET /api/air-conditioners`	ทุกคน	—	คืนรายการแอร์ทุกเครื่อง: `deviceId` , `roomNumber` (`ir` / `wired`), `isOnline` , `pzemAvailable` `energy` , `state` , `autoOffSecondsRemaini`
`POST /api/air-conditioners/:id/command`	ทุกคน	`{"power":true,"temperature":25,"mode":"cool","fanSpeed":"auto","swing":"off",...}`	ตอบ `202 Accepted` พร้อม `{"commandId":".` `/command` ทันที
`POST /api/air-conditioners/reservations`	Admin	`{"deviceId":"ESP32-...","bleName":"RMUTT-AC-...","roomNumber":"16301","airNumber":1,"protocol":"DAIKIN"}`	สร้างบัญชี MQTT บน Mosquitto แล้วคืน `{"mqttHost","mqttPort","mqttUsername",` ทาง BLE
`GET /api/energy/summary`	ทุกคน	Query: `?period=day|week|month|custom&from=YYYY-MM-DD&to=YYYY-MM-DD&floor=3`	คืน `totals` (`kwh` , `previousKwh` , `change` `rooms` (ยอดรายห้อง), `devices` (ยอดรายเครื่อง)
`GET /api/energy/history` `GET /api/energy/export`	ทุกคน	Query: `scope=building|room|device` , `detail` , `from` , `to` , `metrics` , `rate`	`/history` คืนตารางข้อมูล JSON ส่วน `/export`
`GET /api/alerts` `POST /api/alerts/:id/acknowledge`	ทุกคน	Query: `?status=open&limit=30`	คืนรายการแจ้งเตือน (`outside_schedule_occupi` และจำนวนที่ค้างอยู่ (`counts`)

`GET /api/schedules`
`POST /api/schedules/import/preview`
`POST /api/schedules/import/commit`
	อ่าน: ทุกคน
	แก้: Admin
		ส่งไฟล์ `.xlsx` หรือ `.csv` ของตารางสอน และเลือกโหมดนำเข้า (`replaceAll: true/false`)	แกะชีต Excel ทุกห้อง ตรวจสอบวิชาซ้อนทับ แสดงตัวอย่า

`GET / PUT / DELETE /api/schedules/terms` `GET / POST / PUT / DELETE /api/bookings`	อ่าน: ทุกคน แก้: Admin	วันเปิด–ปิดภาคเรียน (เช่น `{"semester":"1/2569","startDate":"2026-06-01","endDate":"2026-10-15"}`) และการจองห้องพิเศษ	ใช้กำหนดว่าวันไหนเปิดหรือปิดภาคเรียน และช่วงเวลาไหน

`GET /api/settings`
`PATCH /api/settings/auto-off`
`PATCH /api/settings/rate`
	อ่าน: ทุกคน
	แก้: Admin
		• `PATCH /auto-off` : `{"enabled":true}` หรือ `{"testMode":true}`
		• `PATCH /rate` : `{"rate":4.50}`
			คืน
			`{"autoOffEnabled":true,"autoOffTestMod`
			`{...}}` (เมื่อสลับ `testMode` จะรีเซ็ตตัวนับทุกห้อง

`GET / POST / DELETE /api/firmware` `POST /api/air-conditioners/:id/ota`	Admin	อัปโหลดไฟล์ไบนารี `.bin` (Header `X-File-Name`) หรือสั่งอัปเดตด้วย `{"fileName":"..."}`	ตรวจ Magic Byte `0xE9` ของ ESP32 ตรวจชนิดบอร์

ส่วนที่ 6 ลำดับการทำงานตั้งแต่ต้นจนจบในสถานการณ์จริง (End-to-End Scenarios)

🎬 สถานการณ์ที่ 1: ผู้ใช้กดปุ่ม "เปิดแอร์ 25°C โหมด Cool" ในแอปมือถือ หรือเว็บแดชบอร์ด

1. **แอปมือถือส่งคำขอ:** เมื่อผู้ใช้กดปุ่มส่งคำสั่ง วิดเจ็ต `AirControlCard` เรียก `AirConditionerService.sendCommand()` ยิง `POST /api/air-conditioners/ESP32-XXXX/command` พร้อมแนบ JWT Token และ JSON `{"power":true,"temperature":25,"mode":"cool","fanSpeed":"auto","swing":"off"}`
2. **Backend ตรวจสอบ 3 ชั้น:** Middleware `authenticate.js` ตรวจลายเซ็น JWT ⟶ `routes/air-conditioners.js` ตรวจว่าบอร์ดลงทะเบียนแล้ว (`provision_status = 'active'`) ⟶ `validateAirState()` ตรวจความถูกต้องตามยี่ห้อแอร์ (เช่น Saijo Denki รับ 18–29°C, Daikin สายแอร์รับเฉพาะฟังก์ชันที่รีโมตผนังมี)
3. **Backend ส่งต่อเข้า MQTT และตอบ 202:** Backend สร้างรหัส `commandId` แล้ว Publish เข้า Topic `rmutt/ac/ESP32-XXXX/command` (QoS 1) จากนั้นตอบ `202 Accepted` กลับให้แอปทันที
4. **บอร์ด ESP32 รับคำสั่งและสั่งแอร์:** ฟังก์ชัน `mqttCallback()` บนบอร์ดรับข้อความมาเก็บไว้ แล้ว `loop()` เรียก `processMqttCommand()` บันทึกสถานะลงหน่วยความจำ NVS และยิงแสงอินฟราเรด 38 kHz (หรือส่งเฟรม SLM8 เข้าสายแอร์) ไปยังเครื่องแอร์
5. **บอร์ดรายงานผลกลับ:** บอร์ดส่งสถานะหน้าปัดรีโมตกลับเข้า `rmutt/ac/ESP32-XXXX/state` และเมื่อคอมเพรสเซอร์แอร์เริ่มเดินจน PZEM วัดกำลังไฟได้ `≥ 30 W` ติดกัน 3 ครั้ง บอร์ดจะส่ง `acRunning: true` พร้อมค่าวัตต์จริงเข้า `rmutt/ac/ESP32-XXXX/status`
6. **แอปแสดงผล:** รอบดึงข้อมูลทุก 10 วินาทีของแอป (`GET /api/rooms` และ `/api/air-conditioners`) จึงเห็นว่าแอร์เปิดทำงานและแสดงกำลังไฟวัตต์จริงบนจอ

🎬 สถานการณ์ที่ 2: ห้องว่างไม่มีคน ⟶ ระบบนับถอยหลังและตัดแอร์อัตโนมัติ (โหมดปกติ vs โหมดทดสอบ)

1. **เรดาร์ตรวจพบว่าคนออกจากห้อง:** บอร์ด `OCC32-XXXX` อ่านค่า LD2410 ทุก 250 ms เมื่อทุกเกต (0–8) มีค่า `staticEnergy <= 15` บอร์ดจะส่ง `{"occupied":false,"radarAvailable":true}` เข้า Topic `rmutt/occupancy/OCC32-XXXX/status` ทันที
2. **Backend ประเมินกติกา 5 ด่าน (`decideAutoOff`):**
   - **ด่าน 0:** สวิตช์รวมทั้งอาคาร (`auto_off_enabled`) เปิดอยู่หรือไม่?
   - **ด่าน 1:** ห้องนี้มีเซนเซอร์ ออนไลน์อยู่ และ `radarAvailable === true` หรือไม่? (ถ้าเซนเซอร์หลุด ⟶ ไม่ปิดแอร์)
   - **ด่าน 2:** มิเตอร์ไฟแอร์ `pzemAvailable === true` หรือไม่?
   - **ด่าน 3:** ห้องว่างจริงไหม (`occupied === false`)? (ถ้ามีคนเดินกลับเข้ามา ⟶ ยกเลิกการนับทันที)
   - **ด่าน 4:** แอร์กำลังกินไฟอยู่จริงไหม (`acRunning === true`)?
   - **ด่าน 5 (เริ่มนับถอยหลัง):** ตรวจตารางเรียน + ปฏิทินเปิด–ปิดเทอม + การจองห้อง ว่าตอนนี้อยู่ในคาบเรียนหรือไม่ (`withinClass`) และตรวจว่าเปิด **โหมดทดสอบ (`testMode`)** อยู่หรือไม่:
     - **โหมดปกติ (`testMode = false`):** นอกคาบเรียน นับ **300 วินาที (5 นาที)** · ในคาบเรียน นับ **900 วินาที (15 นาที)**
     - **โหมดทดสอบ (`testMode = true`):** นอกคาบเรียน นับ **30 วินาที** · ในคาบเรียน นับ **60 วินาที (1 นาที)**
3. **แสดงเวลานับถอยหลังบนแอปและเว็บ:** Backend บันทึก `auto_off_deadline_at` ลงตาราง `air_conditioners` แอปและเว็บที่ดึงข้อมูลอยู่จึงแสดงป้ายนับถอยหลังแบบเรียลไทม์บนการ์ดห้องและการ์ดแอร์
4. **เมื่อครบกำหนดเวลา:** บริการ `services/auto-off.js` ส่งคำสั่ง `{"power":false,"source":"auto-off"}` ไปยัง `rmutt/ac/ESP32-XXXX/command`
   - หากผ่านไป 20 วินาทีแล้วแอร์ยังกินไฟอยู่ จะยิงซ้ำทุก **20 วินาที สูงสุด 5 ครั้ง** (ถ้าครบ 5 ครั้งยังไม่ดับ จะสร้างใบแจ้งเตือน `auto_off_failed`)
   - เมื่อกำลังไฟลดลงจน `acRunning = false` ระบบจะล้างนาฬิกาและสร้างใบแจ้งเตือน `auto_off_succeeded` แอปมือถือที่ดึง `GET /api/alerts` จะเด้ง Notification แจ้งเตือนขึ้นบนจอมือถือทันที

🎬 สถานการณ์ที่ 3: แอดมินสลับสวิตช์ "โหมดทดสอบการปิดอัตโนมัติ (30 วิ / 1 นาที)"

1. **แอดมินกดสวิตช์ในหน้าตั้งค่า (แอปมือถือ หรือ เว็บแดชบอร์ด):** ส่งคำขอ `PATCH /api/settings/auto-off` พร้อม JSON `{"testMode":true}`
2. **Backend บันทึกและรีเซ็ตตัวนับทันที:**
   - บันทึก `auto_off_test_mode = '1'` ลงตาราง `system_settings` และล้าง Cache การตั้งค่า
   - เรียก `clearAllCountdowns()` เพื่อล้าง `auto_off_deadline_at` เดิมของทุกเครื่อง (ที่อาจกำลังนับค้างไว้ที่ 5 หรือ 15 นาที)
   - เรียก `evaluateAutoOff()` ทันทีในจังหวะเดียวกัน ทำให้ห้องที่ว่างอยู่เริ่มนับถอยหลังใหม่ที่ **30 วินาที (นอกคาบ)** หรือ **1 นาที (ในคาบ)** ทันทีโดยไม่ต้องรอรอบประเมินถัดไป

🎬 สถานการณ์ที่ 4: การคำนวณค่าไฟรายชั่วโมง และการเก็บข้อมูลย้อนหลังเมื่ออินเทอร์เน็ตหลุด

1. **กรณีออนไลน์ปกติ:** ทุกครั้งที่บอร์ดส่ง `/status` (ทุก 10 วินาที) `energy-recorder.js` จะนำเลขมิเตอร์สะสม `energy` ล่าสุด ลบด้วยเลขมิเตอร์ครั้งก่อนหน้า (`energy_chain_kwh`) ได้เป็นหน่วยไฟที่เพิ่มขึ้น แล้วบวกสะสมลงในตาราง `energy_hourly` ของชั่วโมงนั้น (ตามเวลาประเทศไทย `Asia/Bangkok`)
2. **กรณีอินเทอร์เน็ตหลุดหรือเซิร์ฟเวอร์ดับ:** บอร์ดแอร์ยังคงอ่าน PZEM ทุก 2 วินาที และทุก ๆ **5 นาที** จะสรุปเลขมิเตอร์และเวลาทำงานเป็นเรคคอร์ดขนาด 32 ไบต์ บันทึกลงพาร์ติชัน Flash ชื่อ `rmuttlog` (ขนาด 64 KB เก็บได้ 2,048 รายการ หรือประมาณ **7 วันเต็ม**)
3. **เมื่ออินเทอร์เน็ตกลับมา:** บอร์ดจะสุ่มรอเวลา 5–45 วินาที (ป้องกันบอร์ดทั้งตึกยิงพร้อมกัน) แล้วส่งข้อมูลย้อนหลังเข้า Topic `/history` ทีละ 10 รายการ รอจนกว่า Backend จะบันทึกและตอบ `/history-ack` กลับมา จึงลบชุดนั้นออกจาก Flash ทำให้ข้อมูลการใช้ไฟไม่สูญหายแม้ไฟหรือเน็ตดีกจะมีปัญหา